Paper VIII
Fifth Lean Verification Block

Packet Operator Algorithms, Exact Compilation,
and GPU Realization

Komorebi P. September 6, 2026

Scope and result

This supplement continues the strengthened manuscript dated August 14, 2026 and the checked fourth block. The original manuscript and its source package remain unchanged. All **23 upstream proof modules** and four earlier mathematical supplements are preserved byte for byte.

Nine additional manuscript-proved items are covered. The finite register is now **55 of 56**, with small-core optimal recompression remaining. Four classical inputs are tracked separately. This block adds **66 theorem declarations in seven modules**, giving **313 declarations across 30 modules**. Of these, 294 belong to Paper VIII and 19 are in its unchanged reused quantum-packet foundation. Supporting lemmas are included, not counted as independent discoveries.

New area	Source items	Declarations
Constructed levels and optimal task depth	6.6, 10.3	17
Actual staged triangular solve	6.5	7
Inverse support through reachable subspaces	6.7	9
Schur support through hidden paths	6.8	4
Banded addresses, storage and work	4.1	9
Exact block-discovery residual	9.2	8
Block-Jacobi condition and CG factor	7.4, 7.6	12
Total added	Nine named items	66

All 30 modules passed a fresh cumulative build with warnings treated as errors. The 313 theorem audits and 36 separately counted construction audits have only the allowed foundational dependencies, or subsets thereof. Both invalid-proof controls were rejected. Seventeen verifier tests and twelve new release-contract tests passed.

Boundary. Scheduling uses finite acyclic graphs, unit-duration tasks and unlimited concurrency. The algebraic and analytic certificates concern actual mathematical objects. They do not certify GPU kernels or timing. The conjugate-gradient corollary propagates a classical convergence estimate; it does not reprove that external algorithm theorem.

1. Constructing the optimal level schedule

Source items 6.6 and 10.3; module GraphSchedules

Let V be finite and let $u \to v$ mean that task v depends on task u. Assume there is no nonempty directed cycle. A topological order or valid level function is *not* supplied as an additional hypothesis.

For each vertex, define its strict ancestor set

$$\mathrm{Anc}(v) = \{u : \text{there is a positive-length path } u \rightsquigarrow v\}.$$

For every edge $u \to v$, the proof establishes

$$\mathrm{Anc}(u) \subsetneq \mathrm{Anc}(v).$$

The inclusion follows by appending the edge. It is strict because u is an ancestor of v but cannot be its own strict ancestor in an acyclic graph. Cardinality therefore decreases when following predecessors. This constructs a well-founded relation on which the level recursion is defined:

$$\boxed{\ell(v) = 1 + \max_{u \to v} \ell(u), \qquad \max \varnothing = 0.}$$

Validity and attainment. Lean proves $\ell(v) > 0$ and $\ell(u) < \ell(v)$ on every edge. A path of n edges from a to b satisfies

$$\ell(a) + n \le \ell(b).$$

Conversely, predecessor-max attainment and induction produce an actual path with $\ell(v) - 1$ edges ending at each vertex v. The result is an equality with a longest path, not merely an upper bound.

A valid schedule $s : V \to \mathbb{N}$ assigns positive stages and strictly increases on each edge. Induction proves the stronger pointwise statement

$$\ell(v) \le s(v) \quad \text{for every } v.$$

Consequently the constructed schedule achieves the least possible makespan,

$$\boxed{d = \max_{v \in V} \ell(v).}$$

For a nonempty graph, d equals the number of vertices on an attained longest path. Vertices of the same level have no dependency edge in either direction.

Empty graph and interpretation. When $V = \varnothing$, the checked depth is zero. The phrase "one plus the longest edge-path length" applies to nonempty graphs. These are ideal discrete scheduling statements. Weighted task costs, a finite processor budget, launch latency and communication are not part of this model. The definitions are mathematical constructions, not a certified hardware scheduler.

2. Forward substitution with an actual stage evaluator

Source item 6.5; module TriangularSolve

Use the graph and levels constructed on the previous page. A vertex v carries an arbitrary vector space X_v over a field. The spaces may have different dimensions. Let

$$D_v : X_v \xrightarrow{\sim} X_v, \qquad T_{vu} : X_u \longrightarrow X_v$$

be an invertible diagonal map and linear transfer maps. The operator being solved is defined by its actual block action:

$$(Ax)_v = D_v x_v + \sum_{u \to v} T_{vu} x_u.$$

No solution, solve equation or precomputed topological certificate is assumed.

Constructed solution. Well-founded recursion defines

$$\boxed{x_v = D_v^{-1}\left(b_v - \sum_{u \to v} T_{vu} x_u\right).}$$

Applying D_v proves the original equation at v. If another vector satisfies all equations, induction over predecessors makes each previously used coordinate identical, and invertibility of D_v forces agreement at v. Thus the construction gives existence and uniqueness, and the block operator is bijective.

Parallel stages. A second, independent definition is an explicit staged evaluator. Set $x^{(0)} = 0$ and

$$x_v^{(k+1)} = \begin{cases} D_v^{-1}\left(b_v - \sum_{u \to v} T_{vu} x_u^{(k)}\right), & \ell(v) = k+1, \\ x_v^{(k)}, & \ell(v) \neq k+1. \end{cases}$$

It reads the previous-stage vector, not partially updated coordinates. The invariant is

$$\ell(v) \leq k \quad \Longrightarrow \quad x_v^{(k)} = x_v.$$

Every predecessor of a level-$k+1$ vertex has level at most k, so the recurrence uses only coordinates already proved correct. At the canonical depth d this yields

$$\boxed{Ax^{(d)} = b.}$$

The local independence theorem separately proves that changing coordinates outside the strict earlier levels cannot change a vertex's local solve.

Scope. Diagonal invertibility is expressed by genuine linear equivalences. The task graph may include declared transfers whose values happen to be zero; optimality refers to that declared dependency graph. Exact field arithmetic and exact local inverses are used. No floating-point stability or implementation-specific parallel speed is inferred.

3. Support of inverses and Schur elimination

Source items 6.7 and 6.8; modules InverseSupport and SchurSupport

Let (P_a) be a complete idempotent packet family in a finite-dimensional algebra. Define $a \rightsquigarrow b$ by an actual finite path through nonzero blocks P_bAP_a. **Zero-length paths are included**, so every packet reaches itself.

Fix the source packet a. In the regular representation, define the submodule

$$W_a = \{x : P_b x = 0 \text{ whenever } a \not\rightsquigarrow b\}.$$

It contains P_a. Completeness of the projectors expands each component of Ax into packet transfers. If a transfer into an unreachable packet were nonzero from a reachable packet, it would extend a path, a contradiction. Thus $AW_a \subseteq W_a$.

Suppose $BA = I$. Left multiplication by A is injective, including on W_a. Finite-dimensionality makes that restricted map surjective. Given $x \in W_a$, choose $y \in W_a$ with $Ay = x$. Then $Bx = BAy = y \in W_a$. Applying this to P_a proves

$$\boxed{a \not\rightsquigarrow b \quad \Longrightarrow \quad P_bBP_a = 0.}$$

The matrix specialization takes B to Mathlib's actual nonsingular matrix inverse, with an invertible determinant. No inverse-sparsity hypothesis is inserted.

Hidden elimination. For the actual Schur complement

$$S = A_{RR} - A_{RH}A_{HH}^{-1}A_{HR},$$

the hidden inverse is expanded through the hidden projectors. A nonzero retained correction block requires indices j, k with nonzero entrance and exit blocks and a hidden path $j \rightsquigarrow k$. Therefore a nonzero block S_{ba} requires either a direct retained block, or

$$a \longrightarrow j \ \rightsquigarrow_H \ k \longrightarrow b.$$

The entrance and exit matrices may be rectangular; retained and hidden coordinate sizes and packet systems are independently typed. The hidden matrix must be invertible. A zero-length hidden path permits entrance and exit through the same hidden packet.

Inclusion is not equality. Cancellation can remove a block permitted by a graph route. For example,

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{has} \quad (A^{-1})_{13} = 0$$

despite a direct nonzero $(1, 3)$ block. The implementation proves support inclusion and the corresponding no-route zero certificate, not a converse.

4. Exact banded addresses and modeled work

Source item 4.1; module BandedCount

For p packets and half-bandwidth w, the actual address set is

$$\mathcal{B}_{p,w} = \{(i,j) \in \{0,\dots,p-1\}^2 : i \le j+w \text{ and } j \le i+w\}.$$

The predicate is the integer version of $|i-j| \le w$. The count is first defined as a double sum of indicators, then identified with the cardinality of this subtype. It is not defined by the target closed form.

Under $0 \le w < p$, the formal proof establishes

$$\boxed{N_{p,w} = |\mathcal{B}_{p,w}| = p(2w+1) - w(w+1).}$$

The proof uses finite prefix counting, the newly exposed boundary row and column when p increases, and the saturated initial band. An intermediate subtraction-free identity avoids relying on natural-number subtraction without the required inequality. The diagonal case $w = 0$ gives exactly p blocks, and the saturated case $w = p-1$ gives p^2.

Stored scalars. With equal block width b, the storage-address type is

$$\mathcal{S} = \mathcal{B}_{p,w} \times \{0,\dots,b-1\}^2.$$

Its cardinality is the actual number of represented scalar entries:

$$\boxed{|\mathcal{S}| = b^2 N_{p,w}.}$$

Arithmetic events. For q right-hand sides, the declared event type adds a right-hand-side index and one of two operation roles, multiplication or accumulation:

$$\mathcal{O} = \{0,1\} \times \{0,\dots,q-1\} \times \mathcal{S}.$$

Hence

$$\boxed{|\mathcal{O}| = 2qb^2 N_{p,w}.}$$

These identities include zero right-hand-side count. The abstract storage formula also admits zero block width. To use the manuscript's substitution $p = n/b$, one must separately require $b > 0$ and $n = pb$.

Boundary. Two events per coefficient and right-hand side are a declared arithmetic convention. These counts do not identify the number of CUDA instructions, account for fused multiply-add lowering, price complex arithmetic in real operations, or include allocation, indexing, communication and launch overhead. They are also not a performance comparison with dense multiplication.

5. Exact residual certificates for supplied packets

Source item 9.2; module DiscoveryCertificate

Let (P_a) be a complete orthogonal projector system on a finite complex coordinate space. The projectors need not be coordinate masks. For an actual matrix A, define

$$\Pi(A) = \sum_a P_a A P_a, \qquad E = A - \Pi(A).$$

Reconstruction gives the exact off-diagonal residual

$$E = \sum_{b \neq a} P_b A P_a.$$

Hilbert–Schmidt orthogonality. For distinct ordered packet pairs $(b, a) \neq (d, c)$, adjoint multiplication and projector orthogonality give

$$\operatorname{tr}\big((P_b A P_a)^* P_d B P_c\big) = 0.$$

If $b \neq d$, the central projector product vanishes. Otherwise $a \neq c$, and cyclicity of trace brings $P_c P_a$ together. This is proved for arbitrary input matrices A, B.

Lean identifies the actual Frobenius norm with its trace expression,

$$\|M\|_{\mathrm{F}}^2 = \operatorname{Re}\operatorname{tr}(M^* M),$$

then expands the finite sum and removes each cross term. The result is

$$\boxed{\|A - \Pi(A)\|_{\mathrm{F}}^2 = \sum_{b \neq a} \|P_b A P_a\|_{\mathrm{F}}^2 .}$$

No Pythagorean equality for the target residual is assumed.

Operator norm. In a separate scoped matrix norm, the triangle inequality gives

$$\boxed{\|A - \Pi(A)\|_2 \leq \sum_{b \neq a} \|P_b A P_a\|_2 .}$$

Here $\|\cdot\|_2$ is the Euclidean induced operator norm, not the Frobenius norm. The formal declarations use distinct norm instances, preventing the two certificates from being conflated. Self-adjointness is used for Frobenius orthogonality; the sum estimate itself requires only the complete packet decomposition.

Discovery boundary. This result certifies the residual of a *supplied* decomposition. It does not construct useful projectors from an arbitrary matrix, prove that a searched decomposition is optimal, or guarantee that its application is profitable on a GPU. The simultaneous spectral-decomposition input remains separately recorded as classical.

6. Block-Jacobi normalization and the CG factor

Source items 7.4 and 7.6; module BlockJacobi

Work on a nonempty finite complex coordinate space with Euclidean operator norms. Assume D and $A = D + K$ are positive definite. Define

$$S = (\sqrt{D})^{-1}, \qquad B = SAS, \qquad E = SKS.$$

The square root is the actual positive matrix square root supplied by continuous functional calculus; the inverse is the actual matrix inverse. Positive definiteness makes $\sqrt{D}$ a unit. Its square identity and inverse identities prove

$$SDS = I, \qquad B = I + E.$$

A direct inverse bound. If $C = (I + E)^{-1}$ and $\|E\|_2 \le \eta < 1$, then $C = I - EC$. Consequently

$$(1 - \eta)\,\|C\|_2 \le 1, \qquad \|I + E\|_2 \le 1 + \eta.$$

The constructed normalization is invertible because S and A are. Combining the estimates gives the actual condition number bound

$$\boxed{\kappa_2(B) := \|B\|_2 \left\|B^{-1}\right\|_2 \le \frac{1+\eta}{1-\eta}.}$$

This norm argument does not require a separately assumed Hermitian perturbation. Nonnegative η follows from the norm bound. Nonemptiness is explicit because the condition-number and norm-of-identity convention uses a nonzero coordinate space.

The preconditioning corollary. Define

$$q(\kappa) = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}.$$

Monotonicity of square root and positive denominators prove that q is monotone. Lean also proves $\kappa_2(B) \ge 1$, $q(\kappa) \ge 0$ for $\kappa \ge 1$, and $q(\kappa) < 1$. Thus

$$\boxed{q(\kappa_2(B)) \le q\left(\frac{1+\eta}{1-\eta}\right).}$$

For $1 \le \kappa$ and $e_0 \ge 0$, an explicit classical premise $e_m \le 2q(\kappa)^m e_0$ and a certified upper bound $\kappa \le \bar{\kappa}$ yield $e_m \le 2q(\bar{\kappa})^m e_0$.

What is not claimed. The classical conjugate-gradient convergence theorem, item 7.5, is not newly formalized. The new corollary proves its factor comparison and bound substitution. It does not replace the original theorem's algorithm hypotheses with a new axiom or claim a floating-point CG convergence proof.

7. Verification record and remaining boundary

Cumulative fifth-block evidence

Fresh Lean checks. A separate baseline rebuild accepted the unchanged 23-module, 247-declaration fourth block. The final run rebuilt all 30 active modules into a new output directory under Lean 4.34.0-rc2 and Mathlib revision

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`.

All modules returned exit code zero with warnings treated as errors. All 313 theorem and 36 construction audit targets matched the inventory. Dependencies are contained in `propext`, `Classical.choice` and `Quot.sound`. Both deliberately invalid controls were rejected. Seventeen existing verifier tests and twelve new release-contract tests passed.

Preservation and corrections. All 23 upstream proof files and the original manuscript assets are unchanged. The source-to-result ledger is updated, including the explicit conventions for empty graphs, reflexive reachability and nonempty condition-number spaces. Two stale fourth-ledger increment fields were corrected: they had retained older "new items" metadata even though its cumulative count was correct. The new regression tests check these fields against this block's actual targets.

Development failures. Seventeen captured fifth-block development checks contain ten failed drafts and seven accepted drafts, one per new module. Failures included declaration scoping, finite-type instances, rewrite direction, parser syntax and warnings. The source snapshots and logs are retained separately. They are not substituted for the successful fresh cumulative run. Every final active source has a matching successful audit.

Independent mirrors. Exact rational tests passed 820 band-address cases, 200 DAG/staged-solve cases, 60 inverse-support cases, 50 Schur-support cases and 72 non-coordinate projector cases. Of the DAG cases, 125 also exhausted all candidate schedules on at most four vertices. Eight cyclic graphs were rejected. The inverse and Schur tests checked 390 and 97 absent-route blocks respectively, and retained a cancellation control.

Separate numerical tests passed 64 complex projector cases and 120 positive-matrix normalization/CG-factor cases. The largest squared-Frobenius identity residual was 1.42×10^{-13}; the largest positive condition-bound violation was 6.66×10^{-15}, within the declared tolerance. These are independent Python mirrors, not Lean execution, new universal proofs or GPU measurements.

Remaining item. Manuscript theorem 5.6, small-core optimal recompression, remains unformalized. Four external inputs remain separately tracked: Eckart–Young–Mirsky, CG convergence, the floating-point product model and simultaneous spectral decomposition. The register's 55/56 therefore must not be read as verification of every assertion or all classical algorithms in the paper.

Reproduction and limits. Run `scripts/verify.py` with the separately supplied portable kit. The current evidence is `verification/fifth_final/SUMMARY.json`. The original CPU/GPU corpus, earlier compiler execution suites and Papers I–VII in full were not rerun. The pinned Mathlib cache was reused, not independently rebuilt or rechecked from source. No manuscript-to-formal semantic-equivalence theorem is claimed.